

1. Part I – Ingestion Layer: Kafka Cluster Sizing

Context

The Kafka cluster has $N = 7$ brokers. Each broker has an independent probability $p = 0.01$ of being unavailable during any given hour. The `orders` topic has $P = 16$ partitions and replication factor $r = 3$ with `min.insync.replicas = 2`. Orders are keyed by `customer_id`.

1.1 Q1 – Hash partitioning and load balancing

During a one-hour window, the platform receives $n = 4\,800\,000$ orders. Assume `customer_id` values are uniformly distributed (i.e. perfect hashing).

- (a) Compute the expected number of orders per partition $\mathbb{E}[X_i]$ and the standard deviation $\sigma(X_i)$.
- (b) Compute the coefficient of variation $\sigma(X_i)/\mathbb{E}[X_i]$ and comment on the quality of the load balance.
- (c) During the Black Friday sale, 25% of all orders come from a single corporate buyer (B2B account). What is the expected load of the partition holding that buyer's orders, and how does it compare to the average? What practical problem does this create?

$$(a) \quad p = \frac{1}{P} = \frac{1}{16} \quad \therefore \mathbb{E}[X_i] = n \cdot p = 4\,800\,000 \times \frac{1}{16} = 300\,000$$

$$\sigma(X_i) = \sqrt{n \cdot p \cdot (1-p)} = \sqrt{4\,800\,000 \times \frac{1}{16} \times \frac{15}{16}} = \sqrt{281\,250} \approx 530.33$$

$$(b) \quad \sigma(X_i) / \mathbb{E}[X_i] = \frac{530.33}{300\,000} \approx 0.001768 \approx 0.177\%$$

The coefficient of variation is extremely small (0.177%). This means the deviation is negligible compared to the mean. Therefore, the quality of the load balance is excellent.

(c). Expected load calculation:

Orders from the single buyer: $4800000 \times 25\% = 1200000$

Because messages are keyed by customer-id, all 1200000 orders will go to exactly one partition.

Remaining orders: $4800000 \times 75\% = 3600000$

Average load of remaining orders per partition: $3600000 / 16 = 225000$

Total expected load of that specific partition

$1200000 + 225000 = 1425000$ orders.

Comparison to Average $1425000 / 300000 = \underline{4.75} \times$

Practical problem: This create a classic "Data Skew" or "Hot Partition" problem.

The specific kafka broker hosting this partition's leader will suffer from severe resource bottlenecks. This single broker will become overwhelmed, potentially leading to increased latency, message backlogs, or even node failure / crashing.

1.2 Q2 – Replication and reliability

- (d) Compute the probability that a single partition is *lost* during a one-hour window (i.e. all $r = 3$ replicas unavailable simultaneously).
- (e) Compute the probability that a single partition becomes *read-only* (writes blocked because fewer than `min.insync.replicas` are up, but data is not lost).
- (f) The `orders` topic has $P = 16$ partitions. Using the union bound, give an upper bound on the probability that *at least one partition* is lost in a given hour.

$$(d) P(\text{partition lost}) = p^3 = 0.01^3 = 10^{-6}$$

$$(e) P(\text{Read-only}) = P(\text{Exactly } \geq \text{replicas unavailable}) \\ = \binom{3}{2} \cdot p^2 \cdot (1-p) = 3 \times 0.01^2 \times 0.99 = 0.000297$$

$$(f) P(E_i) = 10^{-6} \quad P(\text{At least one partition lost}) \leq 16 \times P(\text{single partition lost}) \\ \leq 16 \times 10^{-6} \\ = 1.6 \times 10^{-5}$$

1.3 Q3 – Quorums and consistency

GlobalCart also runs a Cassandra cluster for customer-session storage with $N_c = 5$ replicas per key. The product team needs **strong consistency** on read.

- (g) State the quorum inequality that R and W must satisfy.
- (h) List all integer pairs (R, W) with $1 \leq R, W \leq 5$ that satisfy strong consistency.
- (i) Among those pairs, identify the one that minimises the *maximum* of R and W (best worst-case latency). Justify why this is the standard choice in practice.
- (j) During a network partition that isolates 2 of the 5 replicas, which configurations remain available for reads? For writes? Interpret in terms of the CAP theorem.

(g) $R + W > N = 5 \quad \therefore R + W > 5$

(h)

$R \backslash W$	1	2	3	4	5
1					(1,5)
2				(2,4)	(2,5)
3			(3,3)	(3,4)	(3,5)
4		(4,2)	(4,3)	(4,4)	(4,5)
5	(5,1)	(5,2)	(5,3)	(5,4)	(5,5)

(i) $\max(3,3) = 3$

The pair that minimises the maximum of R and W is $(3,3)$, it provides a balanced workload. the system minimizes the worst-case latency for operations.

(j) Available for Reads $R \leq 3$

(1,5) (2,4) (2,5) (3,3) (3,4) (3,5)

Available for Writes $W \leq 3$

(3,3) (4,2) (4,3) (5,1) (5,2) (5,3)

Available for Both $R \leq 3$ $W \leq 3$

Only (3,3)

CAP theorem interpretation: The system must trade off Availability (for writes, if $W > 3$) to maintain strict. Consistency during a network partition (CP over AP)

1.4 Q4 – Little's Law and in-flight events

- (k) The producer rate is $\lambda = 50\,000$ events/second and the average end-to-end latency through Kafka is $L = 40$ ms. Using Little's Law, compute the average number of events $\bar{N}$ *in flight* at steady state.
- (l) Each in-flight event consumes approximately 2 KB of memory. Estimate the total memory budget needed across the producer and broker buffers.

$$\begin{aligned} (k) \quad \bar{N} &= \lambda \cdot L = 50000 \text{ events/second} \times 40 \text{ ms} \\ &= 50000 \text{ events/second} \times 0.04 \text{ seconds} \\ &= 2000 \text{ events} \end{aligned}$$

$$\begin{aligned} (l) \quad \text{Total memory budget} &= \bar{N} \cdot 2 \text{ KB} \\ &= 2000 \times 2 \text{ KB} \\ &= 4000 \text{ KB} \end{aligned}$$

2.1 Q5 – Producer acknowledgements

Each broker has independent failure probability $p = 0.005$ during a single write window. The replication factor is $r = 3$.

- (a) Compute the loss probability for a single record under: `acks=1`; `acks=all` with `min.insync.replicas = 2`; `acks=all` with `min.insync.replicas = 3`.
- (b) At a sustained rate $\lambda = 50\,000$ orders/s, compute the expected number of lost orders per hour under each configuration. Comment on which configurations are acceptable for a financial application.

$$(a) \text{ acks}=1 \quad P(\text{acks}=1) = p = 0.005$$

$$\text{acks}=\text{all with min.insync.replicas}=2$$

$$P(\text{acks}=\text{all}, m=2) = p^2 = 0.005^2 = 0.000025$$

$$\text{acks}=\text{all with min.insync.replicas}=3$$

$$P(\text{acks}=\text{all}, m=3) = p^3 = 0.005^3 = 0.000000125$$

(b) Total orders = $50000 \text{ orders/s} \times 3600 \text{ s} = 180000000 \text{ orders/hour}$

acks=1 Expected loss = $180000000 \times 0.005 = 90000 \text{ orders/hour}$

acks=all with $m=2$

Expected loss = $180000000 \times 0.000025 = 4500 \text{ orders/hour}$

acks=all with $m=3$

Expected loss = $180000000 \times 0.000000125 = 22.5 \text{ orders/hour}$

For a financial application, data loss is completely unacceptable.

acks=1 and acks=all with $m=2$ is entirely unacceptable.

Strictly speaking, acks=all with $m=3$ is unacceptable. But it approaches the ^{necessary reliability} needed.

2.2 Q6 – Retries (geometric model)

A transient send error occurs on each individual attempt with probability $q = 0.03$. Each attempt takes $L_0 = 8$ ms; the exponential back-off between attempts is $B = 100$ ms.

- (c) Compute the expected number of attempts $\mathbb{E}[T]$ to deliver a single record.
- (d) Compute the expected total latency $\mathbb{E}[\text{latency}]$ for delivering one record.
- (e) The producer is configured with `retries` = $R = 4$ (so $R + 1 = 5$ total attempts). Compute the residual loss probability q^{R+1} .
- (f) Combine Q5(a) (`acks=all`, $m = 2$) and Q6(c): what is the overall probability that a record is ultimately lost?

$$(c) \quad \mathbb{E}[T] = \frac{1}{1-q} = \frac{1}{1-0.03} = \frac{1}{0.97} \approx 1.0309 \text{ attempts}$$

(d) Expected Request Latency:

$$\mathbb{E}[T \cdot L_0] = L_0 \cdot \mathbb{E}[T] = 8 \times \frac{1}{0.97} \approx 8.247 \text{ ms}$$

Expected Back-off Latency:

$$\mathbb{E}[2^T - 1] = \sum_{k=1}^{\infty} 2^{k-1} \cdot q^{k-1} \cdot (1-q) = (1-q) \sum_{k=1}^{\infty} (2q)^{k-1} = \frac{1-0.03}{1-2 \times 0.03} \approx 1.03191$$

$$\mathbb{E}[\text{Backoff}] = B \cdot [\mathbb{E}[2^T - 1]] = 100 \times (1.03191 - 1) = 3.191 \text{ ms}$$

$$\therefore t \text{ (latency)} = 8.247 + 3.191 = 11.438 \text{ ms}$$

$$(e) \text{ Residual Loss} = q^5 = (0.03)^5 = 0.000000243 = 2.43 \times 10^{-8}$$

$$(f) \text{ From Q5(a) } P(\text{Hard loss}) = p^2 = (0.005)^2 = 0.000025$$

$$\text{From Q6(e) } P(\text{Transient Loss}) = 2.43 \times 10^{-8}$$

$$P(\text{Ultimate Loss}) = 0.000025 + 0.000000243 - (0.000025 \times 2.43 \times 10^{-8})$$

$$\approx 0.0000250243$$

$$= 2.50243 \times 10^{-5}$$

2.3 Q7 – Delivery semantics

(g) Recall the 2×2 classification of delivery semantics. Fill in the missing cells:

	No duplicates	Possible duplicates
No loss	<u>Exactly - once</u>	<u>At-least-once</u>
Possible loss	<u>At-most-once</u>	(useless)

(h) The consumer is configured with the default behaviour (commit *after* processing). Under at-least-once delivery, the probability a consumer crashes between processing and commit is $\alpha = 2 \times 10^{-4}$ per record. Compute the expected number of times each record is processed.

(i) At $\lambda = 50\,000$ records/s, how many duplicate processings occur per hour on average?

(j) Since each order also triggers a debit on a non-idempotent SQL database, justify mathematically (in one or two sentences) why the team must migrate to exactly-once semantics.

$$(h) \ E(N) = \frac{1}{1-\alpha} = \frac{1}{1-0.0002} = \frac{1}{0.9998} \approx 1.002004$$

$$(i) \text{ Records/hour} = 50000 \times 3600 = 180000000 \text{ records/hour}$$

$$\text{Duplicates} = \frac{1}{1-\alpha} - 1 = \frac{\alpha}{1-\alpha} = \frac{0.0002}{0.9998} \approx 0.00020004$$

$$\text{Total Duplicates} = 180000000 \times \frac{0.0002}{0.9998} \approx 36007.2$$

(j) Because SQL database is non-idempotent, every duplicate processing results in a compounding action. Mathematically, the system will incorrectly double-charge

customers approximately 36000 times per hour.

2.4 Q8 – Consumer-group capacity and lag

The consumer group reading **orders** has C consumers; each consumer processes up to $\mu = 5\,000$ records/s. The topic has $P = 16$ partitions.

- (k) With $C = 8$ consumers, how many partitions does each consumer handle? What is the aggregate consumer throughput? Is the system stable at $\lambda = 50\,000$ records/s?
- (l) A Black Friday spike doubles the producer rate to $\lambda' = 100\,000$ records/s for $T_{\text{spike}} = 60$ s. Compute the lag N_0 accumulated at the end of the spike, assuming $C = 8$ consumers.
- (m) After the spike, the producer rate returns to $\lambda = 50\,000$. Compute the catch-up time $T_{\text{catch-up}}$ needed to drain the backlog N_0 .
- (n) What is the maximum useful number of consumers for this topic? Justify with the effective-parallelism formula.
- (o) If the team wants to sustain $\lambda^* = 120\,000$ records/s with the same μ , what is the minimum number of partitions P^* they should reconfigure the topic with?

$$(k) \frac{P}{C} = \frac{16}{8} = 2 \text{ partitions/consumer}$$

$$\mu_{\text{total}} = C \times \mu = 8 \times 5000 = 40000 \text{ records/s}$$

$$\lambda = 50000 \text{ records/s} > 40000 \text{ records/s} \quad \therefore \text{the system is NOT stable}$$

$$(l) \text{ New accumulation rate } \lambda' - \mu_{\text{total}} = 100000 - 40000 = 60000 \text{ records/s}$$

$$N_0 = 60000 \text{ records/s} \times 60 \text{ s} = 3600000 \text{ records}$$

$$(m) \mu_{\text{total}} - \lambda = -10000 \text{ records/s}$$

It is impossible for the system to catch up. Because the steady-state incoming rate is still strictly greater than the maximum processing capacity 40000, the backlog N_0 will not drain.

$$(A) C_{\text{effective}} = \min(C, P)$$

$P=16$ $\therefore$ maximum useful number of customers is 16.

if $C=17$ $\min(17, 16) = 16$. The 17th customer will sit completely idle.

$$(D) C_{\text{needed}} = \frac{\lambda^*}{\mu} = \frac{120000}{5000} = 24 \text{ consumers}$$

$$\therefore C \leq P \quad \therefore P_{\min} = 24.$$

3. Part III – Processing Layer: Spark Streaming

Context

A Spark Structured Streaming job reads from Kafka and computes a rolling *revenue per country* aggregate. The cluster has $E = 25$ executors with $k = 4$ cores each, and 16 GB memory each.

3.1 Q9 – Spark parallelism and Amdahl's Law

A batch reprocessing job reads $T = 600$ GB of historical data split into $P_0 = 4800$ HDFS blocks of 128 MB.

- (a) Compute the total number of parallel slots S .
- (b) How many waves of parallel tasks are required to process all $P_0 = 4800$ partitions?
- (c) Each task takes $t_{\text{task}} = 8$ s. Compute the total stage time T_{stage} .
- (d) The team plans to coalesce down to $P_1 = 200$ partitions. Assuming task time scales linearly with partition size, compute the new T_{stage} . Is the coalesce useful? Compute the per-task data size for both layouts and identify any concern.
- (e) The job has a parallelisable fraction $f = 0.92$. Compute the speed-up at S slots using Amdahl's Law, and the theoretical maximum speed-up as $S \rightarrow \infty$.
- (f) The team is considering doubling the cluster to $S' = 2S$ slots. Compute the new speed-up and the *relative* improvement compared to S . Comment on whether the doubling is cost-effective.

$$(a) S = Ek$$

$$= 25 \times 4 = 100 \text{ slots}$$

$$(b) \text{ Waves} = \frac{P_0}{S} = \frac{4800}{100} = 48 \text{ waves}$$

$$(c) T_{\text{stage}} = \text{waves} \times t_{\text{task}} \\ = 48 \times 8 = 384 \text{ seconds}$$

$$(d) \text{ Compute new task time } t_{\text{task_new}} = 8 \times 24 = 192 \text{ s}$$

$$\text{Compute new stage time } W_{\text{new}} = 200/100 = 2 \text{ waves } T_{\text{new}} = 192 \times 2 = 384 \text{ s}$$

It is not useful. the total stage time remains exactly the same.

Old per-task data size : 128 MB

New per task data size: $128\text{MB} \times 24 = 3072\text{MB}$

This creates a severe Out-of-Memory (OOM) risk.

If 4 tasks run simultaneously on one executor.

just holding the input partition data requires $4 \times 36\text{B} = 12\text{GB}$.

Only 4 GB for Spark's execution overhead, JVM memory. highly likely causing performance degradation or complete executor crashes.

$$(e) \text{ Speed-up } (100) = \frac{1}{1 - 0.92 + \frac{0.92}{100}} = \frac{1}{0.0892} \approx 11.21x$$

Theoretical maximum speed-up

$$\text{Max Speed up} = \frac{1}{1 - 0.92} = 12.5x$$

$$(f) \text{ Speed up } (200) = \frac{1}{1 - 0.92 + \frac{0.92}{200}} = \frac{1}{0.0846} \approx 11.82x \quad \text{Improvement} = \frac{11.82}{11.21} \approx 1.054$$

Doubling the Cluster size means a 100% increase in hardware costs, but it yields a meager 5.4% improvement in performance. This is highly not cost-effective.

3.2 Q10 – Windowing and watermarks

The streaming job uses a **sliding window** of length $w = 15$ minutes and slide $s = 3$ minutes, with a watermark threshold $\theta = 5$ minutes. There are $K = 25$ countries (one aggregate per country per window). Each window aggregate is 32 bytes.

- (g) How many overlapping windows does each event belong to?
- (h) How many open windows does the engine maintain at any time?
- (i) Estimate the total state memory required (in MB).
- (j) Suppose the team switches to a *tumbling* 15-minute window. By what factor does the state memory shrink?
- (k) Event lateness D is approximately exponential with mean $\mathbb{E}[D] = 60$ s. Compute the rate parameter μ , and the probability that a random event is dropped by the current watermark $\theta = 5$ min.
- (l) The product team requires a drop rate of at most $\alpha = 10^{-4}$. Compute the smallest watermark threshold θ^* that meets this target. Express the result in minutes.

$$(g) \text{ Overlaps} = \frac{w}{s} = \frac{15}{3} = 5 \text{ windows}$$

$$(h) \text{ Open Windows} = \frac{w + \theta}{s} = \frac{15 + 5}{3} = 6.67 \\ = 7 \text{ open window.}$$

$$(i) \text{ Total state} = 7 \text{ windows} \times 25 \text{ countries} \times 32 \text{ bytes/aggregate} = 5600 \text{ bytes}$$

$$(j) \text{ New open windows } \frac{15 + 5}{15} = 1.33 = 2 \text{ open windows} \quad = 0.0056 \text{ MB}$$

$$\text{Factor} = \frac{7}{2} = 3.5$$

$$(k) \mu = \frac{1}{\mathbb{E}[D]} = \frac{1}{60} \text{ s}^{-1} \quad P(D > 300) = e^{-\frac{1}{60} \times 300} = e^{-5} = 0.67\%$$

$$\begin{aligned}
 (b) \quad e^{-\mu \theta^*} &= 10^{-4} & -\frac{\theta^*}{60} &= \ln(10^{-4}) & \theta^* &= -60 \times \ln(10^{-4}) \\
 & & & & & \approx 552.62 \text{ seconds} \\
 & & & & & = 9.21 \text{ minutes}
 \end{aligned}$$

3.3 Q11 – Batch-vs-streaming cost crossover

The streaming job costs $r_s = \$0.30$ per minute to keep alive. A batch alternative would re-run the same aggregation every T minutes at a cost of $C_b = \$12$ per run.

- (m) Compute the refresh-interval threshold T^* at which streaming becomes cheaper than batch.

6

- (n) The product team requires the dashboard to be no more than 10 min stale. Which option is cheaper, and by how much per hour?
- (o) How much money per month (30 days) is saved by the cheaper option, assuming 24/7 operation?

$$(m) \quad T^* \times r_s = C_b \quad T^* = \frac{C_b}{r_s} = \frac{12}{0.3} = 40 \text{ minutes}$$

$$(n) \quad \text{Streaming Cost per hour} = \$0.3/\text{min} \times 60 \text{ min/hour} = \$18 \text{ per hour}$$

$$\text{Batch Cost per hour} = 6 \text{ run/hour} \times \$12/\text{run} = \$72 \text{ per hour}$$

$$\text{Savings} \quad \$72 - \$18 = \$54 \text{ per hour}$$

$$(o) \quad \text{Calculate total hours in a month} = 24 \text{ hours/day} \times 30 \text{ days} = 720 \text{ hours}$$

$$\text{Monthly Savings} = 720 \text{ hours} \times \$54/\text{hour} = \$38880$$

4. Part IV – Storage Layer: Data Lake

Context

Aggregated results are written to an S3 data lake in Parquet format. The raw event archive (one year of clickstream) has $V = 4$ TB uncompressed. The schema has $n = 40$ columns of average width $w = 12$ bytes per row ($W = 480$ bytes/row), with $N = 8.3 \times 10^9$ rows. Parquet uses row groups of $G = 100\,000$ rows.

4.1 Q12 – Column pruning and compression

The dataset is stored in two competing formats:

- CSV + gzip: $r_{\text{row}} = 3$;
- Parquet + Snappy: $r_{\text{col}} = 6$.

A typical query reads $|S| = 6$ columns (out of 40) to compute a country-level KPI.

- Compute the column-pruning ratio ρ_{prune} .
- Compute the bytes read by a CSV scan (compressed).
- Compute the bytes read by a Parquet scan with column pruning (no predicate pushdown).
- Compute the combined speed-up of Parquet over CSV for this query.

$$(d) \text{ Speed-up} = \frac{1.3337 \text{ TB}}{0.1 \text{ TB}} = 13.33 \times$$

$$(a) \rho_{\text{prune}} = \frac{|S|}{n} = \frac{6}{40} = 0.15$$

$$(b) \text{ Bytes Read}_{\text{CSV}} = \frac{V}{r_{\text{row}}} \\ = \frac{4 \text{ TB}}{3} \approx 1.3337 \text{ TB}$$

$$(c) \text{ Bytes Read}_{\text{parquet}} \\ = \frac{V \times \rho_{\text{prune}}}{r_{\text{col}}} = \frac{4 \text{ TB} \times 0.15}{6} = 0.1 \text{ TB}$$

4.2 Q13 – Predicate pushdown

The query also has a filter `WHERE country = 'FR'`. The selectivity is $\sigma = 1/25$ (uniform across countries).

- (e) Compute the probability that a random row group is skipped thanks to its min/max statistics (assume `country_code` values are unsorted).
- (f) Compute the total number of row groups in the dataset.
- (g) Now suppose the data is sorted by `country` before writing. What fraction of row groups must be read?
- (h) Compute the bytes read by Parquet with column pruning *plus* sorted predicate pushdown. Compare to Q12(c).

$$(e) P_{\text{skip}} = (1 - \sigma)^G = \left(\frac{24}{25}\right)^{100000} = (0.96)^{100000} \approx 0$$

$$(f) \text{ Total Row Groups} = \frac{N}{G} = \frac{8.3 \times 10^9}{100000} = 83000 \text{ row groups}$$

$$(g) \sigma = 1/25 \quad \text{'FR' represents exactly } 1/25 \text{ of the total uniform data.}$$

$$(h) \text{ Bytes Read} = 100 \text{ GB} \times \frac{1}{25} = 4 \text{ GB}$$

$$100 \text{ GB} \rightarrow 4 \text{ GB} \quad 25 \times \text{speed-up}$$

4.3 Q14 – Partition design and pruning

The lake is partitioned by country / year / month / day with cardinalities $c_1 = 25$, $c_2 = 3$, $c_3 = 12$, $c_4 = 31$.

- (i) Compute the total number D of leaf directories.
- (j) A typical query reads transactions for France in 2024, between March 1 and March 7 (i.e. 7 days). Compute the partition-pruning ratio ρ_{read} .
- (k) Combine Q13(c) and Q14(b) to compute the overall fraction of the dataset effectively scanned.

i) $D = c_1 \times c_2 \times c_3 \times c_4 = 25 \times 3 \times 12 \times 31 = 27900$ leaf directories

j) Matching Directories = $1 \times 1 \times 1 \times 7 = 7$ directories

$$\rho_{\text{read}} = \frac{7}{27900} \approx 0.0002509 = 0.025\%$$

k) From Q12(a) $\rho_{\text{prune}} = 0.15$ From Q14(j) $\rho_{\text{read}} = \frac{7}{27900}$

$$\text{Overall Fraction} = 0.15 \times \frac{7}{27900} \approx 0.00003763 = 3.763 \times 10^{-5}$$

4.4 Q15 – The small-file problem

S3 has fixed per-file overhead $\tau = 100$ ms and streaming throughput $\beta = 200$ MB/s. Each day-leaf directory currently holds 2 GB of compressed data, split into 250 files of average size 8 MB.

- (l) Compute the optimal average file size s_{opt} . Are the current 8 MB files above or below this threshold?
- (m) For the query in Q14(b) (France, 7 days), how many files F must be read with the current layout?
- (n) Compute the total read time T_{read} for the query under the current layout.
- (o) The team runs a compaction job that rewrites each day-partition as a single 2 GB file. Compute the new total read time and the speed-up achieved.

$$(l) \quad s_{\text{opt}} = \tau \cdot \beta = 0.1 \text{ s} \times 200 \text{ MB/s} = 20 \text{ MB}$$

$8 < 20 \text{ MB}$ the system is suffering from the small-file problem.

$$(m) \quad F = \text{Directories} \times \text{Files per directory} = 7 \times 250 = 1750 \text{ files}$$

$$(n) \quad \text{Overhead} = F \cdot \tau = 1750 \times 0.1 = 175 \text{ s}$$

$$\text{Transfer time} = \frac{\text{Total Volume}}{\beta} = \frac{7 \times 200 \text{ MB}}{200 \text{ MB/s}} = 70 \text{ seconds}$$

$$T_{\text{read}} = 175 + 70 = 245 \text{ seconds}$$

(10) New file count $7 \text{ days} \times 1 \text{ file/day} = 7 \text{ files}$

New Overhead Time $7 \times 0.1 \text{ s} = 0.7 \text{ seconds}$

Transfer Time $T_{\text{new}} = 0.7 \text{ s} + 70 \text{ s} = 70.7 \text{ seconds}$

$$\text{Speed-up} = \frac{T_{\text{old}}}{T_{\text{new}}} = \frac{245}{70.7} \approx 3.47 \times$$

4.5 Q16 – The high-cardinality trap

A product manager proposes adding `user_id` (cardinality 5×10^6) as a partition column between `country` and `year`.

- (p) Compute the new total number of leaf directories.
- (q) Assuming the dataset size remains $V = 4 \text{ TB}$ and each leaf holds at least one file, estimate the average file size in each leaf.
- (r) Justify mathematically why this proposal must be rejected. (Hint: compare to s_{opt} and to the per-leaf metadata cost.)

$$(p) \quad D_{\text{new}} = D_{\text{old}} \times 5 \times 10^6 = 27900 \times 5000000 = 1.395 \times 10^{11} \text{ leaf directories}$$

$$(q) \quad \text{Average file size} = \frac{V}{D_{\text{new}}} = \frac{4 \times 10^{12} \text{ bytes}}{1.395 \times 10^{11} \text{ files}} \approx 28.67 \text{ bytes}$$

(h) $28.67 \text{ bytes} \ll S_{opt} = 20 \text{ MB}$

The system spends 100 ms of overhead just to stream a few dozen bytes. The ratio of actual data transfer time to overhead time approach zero.

5.1 Q17 – Combined throughput budget

Recall: $\lambda = 50\,000$ orders/s, $P = 16$ Kafka partitions, $\mu = 5\,000$ records/s per consumer, $C_{\max} = P$.

- (a) What is the maximum sustainable throughput $\lambda_{\max}$ of the current stack? Where is the bottleneck?
- (b) The marketing team forecasts +60% growth next year. Propose a configuration (new P^* , new C^*) that sustains the forecast. Estimate the cost impact at $r_s = \$0.30/\text{min}$ and a per-consumer rate of $\$0.05/\text{minute}$.

(a) $\lambda_{\max} = C_{\max} \times \mu = 16 \times 5000 = 80\,000 \text{ records/s}$

The bottleneck is the number of Kafka partitions.

The hard limit of 16 partitions prevents the system from ever processing more than 80000 records/s

(b) Target Load = $50000 \times 1.6 = 80000 \text{ orders/s}$

$$C^* = \frac{80000}{5000} = 16 \text{ consumers}$$

Minimum valid configuration $p^* = 16$ $C^* = 16$

$$\text{Current Cost} = \$0.3 + (10 \times \$0.05) = \$0.8 \text{ per minute}$$

$$\text{New Cost} = \$0.3 + (16 \times \$0.05) = \$0.3 + \$0.8 = \$1.10 \text{ per minute}$$

5.2 Q18 – End-to-end reliability budget

The platform has three reliability stages: Kafka producer (Q6(d)), Spark processing (assume $\epsilon_{\text{Spark}} = 10^{-6}$ per record), and S3 writes (assume $\epsilon_{\text{S3}} = 10^{-7}$ per record).

- (c) Assuming the three stages fail independently, compute the end-to-end probability that a single order is lost.
- (d) At $\lambda = 50\,000$ orders/s, what is the expected number of lost orders per year? Comment on whether this meets a 5-nines reliability target (99.999%).

$$\begin{aligned} \text{(c)} \quad P_{\text{loss-total}} &\approx \epsilon_{\text{kafka}} + \epsilon_{\text{spark}} + \epsilon_{\text{S3}} \\ &\approx (2.5 \times 10^{-5}) + (0.1 \times 10^{-5}) + (0.01 \times 10^{-5}) \\ &\approx 2.61 \times 10^{-5} \end{aligned}$$

$$(d) \text{ Total Orders} = 50000 \times \text{Seconds/year} = 50000 \times 365 \times 24 \times 60 \times 60 = 1576800000000$$

$$\begin{aligned} \text{Expected Losses} &= \text{Total Orders} \times P_{\text{loss-total}} \\ &= 1576800000000 \times 2.61 \times 10^{-5} = 41154480 \text{ lost orders per year} \end{aligned}$$

$2.61 \times 10^{-5} > 10^{-5}$ doesn't meet the 5-nines reliability target.

5.3 Q19 – End-to-end latency budget

Each stage contributes to the total user-perceived latency: Kafka in-flight = 40 ms (Q4(a)), Spark micro-batch interval $\Delta t = 5$ s, S3 commit latency = 200 ms.

- (e) Compute the expected end-to-end latency from user click to BI dashboard.
- (f) The product manager demands sub-second latency. Which stage dominates, and what configuration change would you propose first?

$$(e) \text{ Expected Wait} = \underbrace{5000 \text{ ms}}_{\text{Spark}} = 2500 \text{ ms}$$

Kafka: 40 ms S3: 200 ms

$$\text{Expected end to end latency} = 40 + 2500 + 200 = 2740 \text{ ms}$$

- (f) Spark processing stage completely dominates the latency budget.
∴ reconfigure spark streaming job.
 Δt from 5 seconds to sub-second value such as $\Delta t = 500ms$

5.4 Q20 – End-to-end cost-and-storage

The dataset grows by $V_{\text{daily}} = 12$ GB per day uncompressed. Storage costs: S3 at $p = \$0.023/\text{GB-month}$.

- (g) Compute the monthly storage cost in CSV+gzip ($r_{\text{row}} = 3$) vs. Parquet+Snappy ($r_{\text{col}} = 6$) vs. Parquet+Zstd ($r = 10$). Assume a year of accumulated data.
- (h) Across the four layers (Kafka, Spark, S3 storage, S3 access), where should investment focus to reduce total cost? Defend your answer in 3–5 sentences using results from previous questions.

(g) CSV+gzip ($r=3$) $V_{\text{year}} = 12 \text{ GB/day} \times 365 \text{ days} = 4380 \text{ GB}$

Compressed Volume = $4380 / 3 = 1460 \text{ GB}$

Monthly Cost = $1460 \text{ GB} \times \$0.023/\text{GB} = \33.58 per month

Parquet + Snappy ($r=6$)

Compressed Volume = $4380 / 6 = 730$ GB

Monthly cost = $730 \text{ GB} \times \$0.023/\text{GB} = \16.79 per month

Parquet + Zstd ($r=10$)

Compressed volume = $4380 / 10 = 438$ GB

Monthly cost = $438 \text{ GB} \times \$0.023/\text{GB} = \10.07 per month.

(h) Focus on Spark processing layer, and secondarily the Kafka compute layer. From Q11, just keeping the

Spark streaming job alive costs \$0.3 per minute,
\$13000 per month. So. spend on switching from streaming
to batch processing (if SLA allows), turning spark cluster
utilization, or reducing kafka node counts, rather than
squeezing a few extra megabytes out of storage formats.